The SO_KEEPALIVE option for a socket is disabled (set to FALSE) by default. When this socket option is enabled, the TCP stack sends keep-alive packets when no data or acknowledgement packets have been received for the connection within an interval

SO_LINGER setting

The **solinger** parameter sets the delay, in seconds, that the Gateway waits while data is being transmitted before closing a socket, after a call has been received to close the socket.

solinger=<number>
Description

Set the value in the range 0 - 65,536 to specify the delay period in seconds. If a value greater than zero is specified and data is being transmitted when a call to close the socket is received, the Gateway waits until the data is transmitted or until the time specified before closing the socket. If a value is not specified or is set to zero, the socket is closed immediately, terminating the data transmission. However, the data transmission can be successful because TCP/IP repeats the send request for a specified period of time.

This parameter is in the TCP protocol parameters subsection of the GATEWAY section of the configuration file.

This parameter is in the TCP protocol parameters subsection of the GATEWAY section of the configuration file.

Default value

If this parameter is not specified, the default value is 0.

Configuration Tool

In the Configuration Tool, you can set the value of **solinger** in the **SO_LINGER setting** field in the **TCP/IP settings** or **SSL settings** pages.

This option has no meaning for stream sockets. SO_SNDBUF. Allows you to set the size of the send buffer to a value to suit your application needs. The minimum size is 512 bytes and the maximum size is 1048576 bytes.

The SO_RCVBUF socket option determines the size of a socket's receive buffer that is used by the underlying transport. This socket option applies only to listening sockets, datagram sockets, and connection-oriented sockets

"TCP_NODELAY is for a specific purpose; to disable the Nagle buffering algorithm. It should only be set for applications that send frequent small bursts of information without getting an

immediate response, where timely delivery of data is required (the canonical example is mouse movements)." Syntax: on or off

In Java, **BufferedReader** is a class that provides efficient reading of characters from an input stream, such as a file or an input stream. It buffers the input, meaning it reads and stores data in memory in larger chunks, which can improve reading performance compared to reading one character at a time. This class is often used for reading text data from various sources, offering methods like **readLine()** to read lines of text and read() to read individual characters. It's commonly used in conjunction with **InputStreamReader** for reading from input streams.

In Java, **PrintWriter** is a class that provides convenient methods for writing textual data to various output streams, such as files, network sockets, or other destinations. It allows you to write data in a human-readable format, making it easy to create and manipulate text-based files or send text over network connections. **PrintWriter** is commonly used for tasks like writing log files, generating reports, or sending textual data over network protocols. It offers methods like **print()** and **println()** to write formatted text and **write()** to write raw characters or strings to the output stream.

In Java, the `getInetAddress()` method is used to retrieve the IP address associated with a network resource, such as a host name or a network socket. It is typically used with classes like `InetAddress` and `Socket` to obtain the IP address of a remote host or the local machine. This method allows you to work with IP addresses when dealing with network communication, which is essential for tasks like connecting to remote servers or determining the IP address of a network device.

In Java, the `getHostAddress()` method is used to retrieve the IP address in string format associated with an `InetAddress` object. It returns a textual representation of the IP address, which is useful for displaying or working with IP addresses in a human-readable form. This method is commonly used when you need to obtain the IP address of a host or network resource represented by an `InetAddress` object.